Region Resonances

Technical Appendix

Mathematical Concepts Behind the Matching Procedure

Vinotheca • May 2026

This appendix sets out the mathematical concepts behind the Region Resonances matching procedure: the embedding map that converts text to vectors, the geometry it produces, the cosine-similarity score that ranks regions against a query, and the display transformation that converts raw similarities into the percentages users see. The treatment uses standard notation from linear algebra and assumes familiarity with vectors, inner products, and Euclidean geometry, but it requires no specialised background. For the non-technical companion, see *Methodology and Interpretation*.

A. From Text to Vectors

A.1 The embedding map

The pipeline begins with a function that converts text into numerical vectors. Such a function is called an *embedding map*. Formally, if $\mathcal{S}$ denotes the set of finite text strings (passages of arbitrary length, including the user's query and each region's narrative), then an embedding map of dimension d is a function

$$E : \mathcal{S} \longrightarrow \mathbb{R}^d$$

that assigns to each text string s a real vector $E(s) \in \mathbb{R}^d$. In our deployment, the embedding map is OpenAI's `text-embedding-3-small` model, and the output dimension is $d = 1536$.

The embedding model is treated throughout as a fixed, opaque function: we do not concern ourselves with how its weights were learned or how its computation is structured. What matters for the matching procedure is the geometric structure that the map induces on $\mathbb{R}^d$.

A.2 Unit-normalisation

The OpenAI map has a useful property: every output vector is rescaled to have Euclidean length 1. That is, for every input string s,

$$\|E(s)\|_2 \;=\; 1, \qquad \text{where } \|v\|_2 = \sqrt{v_1^2 + v_2^2 + \cdots + v_d^2}.$$

The image of the embedding map therefore lies entirely on the surface of the *unit sphere* in $\mathbb{R}^d$, denoted

$$\mathbb{S}^{d-1} \;=\; \{\, v \in \mathbb{R}^d \,:\, \|v\|_2 = 1 \,\}.$$

Every text the system processes — each of the fifty-nine region passages, every user query — is represented as a point on this sphere. The matching procedure is therefore a question about the geometry of the unit sphere: given a query point and fifty-nine region points, which region points are closest?

A.3 The corpus

Let the corpus consist of $n = 59$ region passages $r_1, r_2, \ldots, r_n$, where each r_i is the concatenation of a region's one-word metaphor with its full Layer-1 identity narrative. The embedding map

produces fifty-nine vectors:

$$\mathbf{r}_i \;=\; E(r_i) \;\in\; \mathbb{S}^{d-1} \qquad (i = 1, \ldots, n).$$

These vectors are computed once, in advance, and stored. They are not recomputed at runtime. The fifty-nine vectors collectively form a fixed reference set against which every user query is compared.

B. Cosine Similarity

B.1 Definition

The standard way to measure similarity between two vectors on the unit sphere is *cosine similarity*, defined for two nonzero vectors $x, y \in \mathbb{R}^d$ as

$$\cos(x, y) \;=\; \frac{\langle x, y \rangle}{\|x\|_2 \, \|y\|_2},$$

where $\langle x, y \rangle = x_1 y_1 + x_2 y_2 + \cdots + x_d y_d$ is the standard inner product. Geometrically, this is exactly the cosine of the angle between x and y when both are regarded as arrows from the origin. It takes values in $[-1, +1]$: the value $+1$ when x and y point in the same direction, 0 when they are perpendicular, -1 when they point in opposite directions.

B.2 The unit-sphere simplification

Because all of our vectors are unit-normalised, the denominator in the cosine-similarity formula is always $1 \cdot 1 = 1$. The formula collapses to

$$\cos(\mathbf{r}_i, \mathbf{q}) \;=\; \langle \mathbf{r}_i, \mathbf{q} \rangle \qquad \text{for } \mathbf{r}_i, \mathbf{q} \in \mathbb{S}^{d-1}.$$

Cosine similarity on the unit sphere is just the inner product. This is the central computational identity of the system: every similarity measurement is one inner-product calculation, no division required.

B.3 Why this is the right notion of similarity

On the unit sphere, three natural ways of measuring how close two points are happen to give the same answer:

- *Cosine similarity* $\langle \mathbf{r}_i, \mathbf{q} \rangle$, ranging from -1 to $+1$, with higher values meaning more similar.
- *Chord distance* $\|\mathbf{r}_i - \mathbf{q}\|_2$, the straight-line distance through the interior of the sphere.
- *Angular distance* $\arccos\langle \mathbf{r}_i, \mathbf{q} \rangle$, the great-circle distance along the surface.

These three quantities are different numbers, but they always agree on which of two candidates is closer to a fixed query: ranking the corpus by descending cosine similarity is identical to ranking it by ascending chord distance, which is identical to ranking it by ascending angular distance. The choice among them is a matter of convenience, not a substantive modelling decision. Cosine similarity is preferred because it is bounded in $[-1, +1]$, dimensionless, and naturally interpretable as "angular alignment."

C. The Matching Procedure

C.1 Computing the similarity vector

When the user submits a query q, the embedding map produces the query vector

$$\mathbf{q} \;=\; E(q) \;\in\; \mathbb{S}^{d-1}.$$

For each region $i = 1, \ldots, n$, the system computes the cosine similarity

$$s_i \;=\; \langle \mathbf{r}_i, \mathbf{q} \rangle.$$

The fifty-nine similarities can be collected in a vector $\mathbf{s} = (s_1, s_2, \ldots, s_n) \in \mathbb{R}^n$. Stacking the region vectors as rows of a matrix

$$\mathbf{R} \;\in\; \mathbb{R}^{n \times d}, \qquad \mathbf{R}_{i,:} = \mathbf{r}_i^{\top},$$

the entire similarity vector is one matrix–vector product:

$$\mathbf{s} \;=\; \mathbf{R}\,\mathbf{q}.$$

C.2 Top-k selection

Given the similarity vector $\mathbf{s}$, the system sorts the indices in descending order of similarity and returns the top k:

$$\mathcal{T}_k(\mathbf{q}) \;=\; \underset{i \in \{1,\ldots,n\}}{\text{top-k}} \; s_i.$$

In deployment, $k = 5$, so the user sees the five regions with highest cosine similarity to the query. The remaining fifty-four are accessible through a *Show all 59 ranked* disclosure.

The geometric content of this procedure is straightforward: the top match is the region whose embedding sits closest to the query embedding on the unit sphere. The second match is the second closest, and so on. The matching procedure is, in this precise sense, a nearest-neighbour search on $\mathbb{S}^{d-1}$.

D. The Display Score

D.1 The empirical range of cosine similarity

Although cosine similarity is mathematically bounded in $[-1, +1]$, the values we actually observe on this corpus occupy a much narrower interval. Across a calibration test set of sixty queries spanning words, phrases, and short prose fragments, the observed similarities s_i fell almost entirely within

$$s_i \;\in\; [\,0.15,\; 0.65\,].$$

Values below 0.15 occur only for pathological inputs (random byte sequences, unrelated languages). Values above 0.70 occur only when the query is itself a paraphrase of a region's source text.

This concentration is a generic feature of high-dimensional embeddings of natural-language text. Almost any two passages of English share a substantial baseline of common semantic structure — they are both grammatical English, they share vocabulary, they refer to a shared world — and the embedding model encodes this shared structure into a common region of the sphere. The signal that distinguishes one passage from another lives in a relatively narrow range above the baseline.

D.2 Why we don't show the raw cosine

Presenting raw cosine values to users would be unflattering and uninformative. A score of 0.62 sounds like a 62% result, but in fact it represents a strong match; a score of 0.31 sounds like failure, but in fact it represents the broadly typical similarity for an unrelated query. Worse, the differences that matter most — the spread between the top match and the tenth match — often occupy only a few hundredths of the cosine range, where they would be visually invisible.

D.3 Clip-and-rescale

To present scores in a range that uses the full visual scale, we apply a piecewise-linear transformation. Define constants $s_{\min} = 0.15$ and $s_{\max} = 0.70$, calibrated against the test set so that the operating range of the cosine corresponds to the operating range of the display. The display score is then

$$\tilde{s}_i \;=\; \mathrm{clip}\!\left(\frac{s_i - s_{\min}}{s_{\max} - s_{\min}},\; 0,\; 1\right) \times 100,$$

where $\mathrm{clip}(t, 0, 1) = \min(1, \max(0, t))$. The transformation maps the empirical operating range $[0.15, 0.70]$ linearly onto $[0, 100]$, and clips anything outside that range to the boundary.

This transformation has three properties that matter for interpretation. It is bounded between 0 and 100, so the displayed scores always fit on a percentage scale. It is monotonically increasing on the operating range, so the displayed ranking is always identical to the underlying cosine ranking. And it is affine on the operating range, so a difference of 10 in the displayed score corresponds to a fixed difference in the underlying cosine, regardless of where on the scale that difference falls. Two regions whose displayed scores differ by 10 are equally distinguishable in the underlying space whether the scores are 40 and 50 or 80 and 90.

D.4 Why not softmax

A common alternative for displaying similarity scores as percentages is the *softmax* transformation, which produces a probability distribution over the corpus:

$$\pi_i \;=\; \frac{\exp(\beta s_i)}{\sum_{j=1}^{n} \exp(\beta s_j)}, \qquad \beta > 0.$$

This also preserves the ranking, but it has two drawbacks for the present application. First, the choice of the temperature parameter β is arbitrary, and different values produce dramatically different displayed probabilities for the same underlying ranking; there is no principled way to set it. Second, softmax is multiplicative in the score gap: the ratio π_i/π_j grows exponentially in $s_i - s_j$. A very small gap in cosine similarity becomes a very large gap in displayed probability, which exaggerates differences between near-tied results and gives users a misleading sense of confidence in the top match.

Clip-and-rescale, by contrast, treats equal cosine differences as equal display differences. Two regions that are nearly tied in true semantic similarity are displayed as nearly tied. This is the honest representation of what the matching procedure has found.

E. What the Mathematics Does and Does Not Establish

The procedure described above is a sequence of well-defined operations: produce a vector via an embedding map, compute inner products against a fixed reference set, sort, transform for display. Each step is mathematically clean and computationally cheap. Given the same query,

the procedure always returns the same ranking, and small changes to the query produce small changes in the similarity vector — both desirable properties.

What the mathematics does not establish is whether the rankings are *good* in the sense of corresponding to human judgment of semantic similarity. That property depends entirely on the embedding map E: whether it has learned, from the text it was trained on, to assign nearby vectors to passages that competent readers would judge similar in meaning. This is not a mathematical question. It is an empirical question about the embedding model, and one whose answer for the OpenAI `text-embedding-3-small` model has been validated by the model's authors and by external benchmarks, but never formally proven.

The mathematical framework presented in this appendix is therefore a guarantee of well-posedness, not of meaningfulness. The procedure will always run, will always produce a ranked list, and will always do so in a way that is geometrically natural and stable under small perturbations. Whether the ranked list reflects something a thoughtful reader of *The Soul of Wine* would recognise as a meaningful resonance is a question that can only be answered by reading the results.

For the non-technical companion, see Methodology and Interpretation.